

**Covariance Matrix Adaptation as an alternative to gradient-based optimization in
reinforcement learning**

Word Count: Unknown

Introduction

Machine learning is a rapidly growing field, focused on the concept of having machines learn to be able to program themselves. As of August 2023, it is a \$158.8 billion market, and between 2022 and 2023 it has grown by 120% (Statista Market Insights, 2023). In machine learning, the machine uses optimizers to optimize models to solve a given problem. My study proposes the usage of a new optimizer "Covariance Matrix Adaptation", to a specific subsection of machine learning called "reinforcement learning". This study seeks to answer the question "To what extent is covariance matrix adaptation an effective alternative to gradient-based optimization methods in actor-critic reinforcement learning?"

Literature Review

Machine learning is an interesting field used in cutting-edge technology to automate problems that would otherwise be difficult to solve by programming only. For example, if we were to try to create a program that identifies the text behind an image of a handwritten note (LeCun et al., 1998), it would be challenging to code for every single case. Machine learning is a field that attempts to solve this problem by creating a program that is capable of "learning" the problem and being able to make its own solutions to the problem. One method in machine learning is known as "deep learning" which is basically a layered linear regression with non-linear functions in between layers.

How deep learning works is simple: we convert the entire image to the grayscale and then convert each pixel on the image to a number between 0 and 1 by inputting the values through an activation function, such as ReLU (Agarap, 2018) or swish (Ramachandran et al., 2017). We then have nodes that take the output of the activation function, multiply it by a constant, and then add a constant. These two constants are called parameters. The output after the fused multiply-add

operation is then fed into another node, which does the same thing but with two different parameters, and the same process repeats itself multiple times until we get an output.

This is called a neural network. Initially, we are not certain of the parameters of each node required to get the desired output. So, what we first need to do is make a function that measures the error of the network, which is measured as the difference between our network's response and the "best possible" response, this function is called the loss function. The job of the "optimizer" is to interpret the relationship between the parameters of the network and the loss of the network to figure out which configuration of parameters results in the lowest possible error. Each of these best-performing configurations of parameters are called "solutions."

In some specific problems though, we do not know the best solution either. If there was a half cheetah with four motors, one on each knee and one on each shoulder, and the goal was to find the exact proportion of torques to get the most horizontal movement, we wouldn't be able to produce an answer because we simply do not know how to calculate it. This is an issue, as it's impossible to calculate the loss of a function if we cannot determine the best possible response. So first, we create an environment for our neural network. The environment is straightforward, it returns "observations" that will be used as the inputs of the neural network, and returns a "score" for the outputs of the neural network. Then we create an agent that trains off of the responses from the environment. This is called reinforcement learning. One method we could use is called vanilla policy gradient, where we flip the optimizer to optimize for highest score rather than lowest error (Williams, 1992). However the issue with vanilla policy gradient is that it converges very slowly and has high variance so instead we use an algorithm in reinforcement learning called TD3 (Fujimoto et al., 2018), in which we make a second neural network, the "critic", that tries to estimate the score value from an observation and an action. The max score value, called

the Q value, is used in the training of the first neural network, the "actor," in determining the error of the actor's outputs, "actions." Now that we have calculated the error, we use it in training the neural network to find the set of parameters with the lowest error. Now, how do we optimize those network parameters? There are 2 categories of optimizers, gradient-based and gradient-free optimizers.

How gradient-based optimizers such as the Stochastic Gradient Descent (SGD) optimizer (Robbins, 1951) work is relatively simple. It calculates the gradient to figure out which direction the gradient goes down. In reality, most of these problems are "black box" optimization problems where we do not know the exact derivative of our gradient. So, we approximate the derivative by finding the average rate of change on the gradient between a minimal distance on the gradient. Based on the direction of the gradient, we make a "jump" in the direction that goes downward and repeat the process. From essential calculus, we know that once the derivative is close to or has reached zero, we have reached the bottom of the gradient. The Adam optimizer (Kingma et al., 2014) has a very similar process, but instead, we use a combination of the gradient and squared gradient to figure out what direction is downward, and it's slightly faster at converging in non-differentiable problems.

There are many different gradient-free optimization methods, but the focus of this research will be on evolutionary strategy-based optimization. Simple Gaussian evolution strategies work by creating a Gaussian distribution with the same number of dimensions as the number of parameters. We then sample (randomly select points on) the multivariate Gaussian distribution to generate a "population" where each dimension in our distribution represents a parameter. Each member of the population most likely has a different value for its parameters than the other members; however, population members are more clumped to the middle of the

range of the parameters (empirical rule of Gaussian distributions). We then search through every population member to find the "elite" members of the population, defined as the members with the lowest error for their parameters. Then we can create a new distribution using the mean and standard deviation of the elites, and treat this new distribution as a new population. From there we can repeat the process of generating a distribution, selecting the elites, and generating new distributions. Each generation of each population should be more performant than the previous generation, and eventually, the mean will converge on a solution.

One major issue we have with evolution strategies though is that our standard deviation is currently a scalar calculated from the standard deviation of the previous generation. The issue with this is that if our algorithm may be confident about one parameter, but is not so confident about the other, the confident parameter weighs the standard deviation down and doesn't let our algorithm "search" for a solution for the not-so-confident parameter. Covariance matrix adaptation (Hansen, N., & Ostermeier, A. 2001) fixes this problem by tracking covariances between the different samples with a covariance matrix. It lets us have a lower standard deviation for our higher confidence parameter while having a higher standard deviation for our lower confidence parameter simultaneously. The algorithm also decouples the standard deviation from the standard deviation from the previous generation, so future generations can search more space than previous generations, allowing for more efficient and faster searching for solutions. We will also refer to covariance matrix adaptation as CMAES.

In 2017, a research group named OpenAI tested the usage of evolution strategies with reinforcement learning problems (Salimans et. al, 2017). They found that in reinforcement learning problems, evolution strategies was able to produce competitive solutions while being much more scalable and cheaper to compute than standard gradient-based optimizers like Adam.

However, since then covariance matrix adaptation has been created, and many improvements have been made to it to reduce its computational complexity. This is what led me to begin my research, and test how effective covariance matrix adaptation is in reinforcement learning problems compared to gradient-based strategies like ADAM.

Method

Before I began my study, I had hypothesized that covariance matrix adaptation would be able to outperform gradient-based strategies like Adam in modern reinforcement learning problems.

To determine whether the usage of covariance matrix adaptation is a competitive solution in machine learning, we need to design an experiment to compare covariance matrix adaptation to gradient-based optimization directly. So we will have 2 experimental groups, the first being the control group which uses Adam, and the second group being the experimental group which uses covariance matrix adaptation. We will evaluate the optimizers in 2 ways, first by the score of the best-performing solution, and second by the time it took to converge on the best-performing solution.

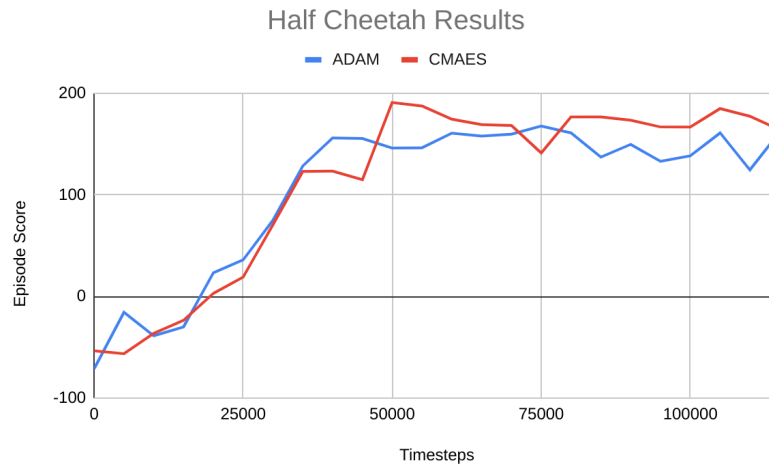
The language that will be used to design the experiments will be Rust. We use Rust because it is a compiled language, which has much greater CPU side efficiency than other traditionally used languages in machine learning. In regular machine learning problems, CPU side efficiency doesn't matter since most of the work is done by the GPU/TPU anyways, but reinforcement learning specifically is very dependent on CPU side efficiency, as the reinforcement learning environment is typically evaluated on the CPU, not the GPU. Rust also has safe bindings to the LibTorch machine learning library and Mujoco physics engine.

Both groups will be using the TD3 agent (Fujimoto et al., 2018). Our implementation will be modified to work with LibTorch and Rust. We block the experiment into 3 groups with 3 different environments, the half cheetah environment (Wawrzyński, 2009), the hopper environment (Erez et al., 2011), and the ant environment (Schulman et al., 2015). These are 3 commonly used environments in machine learning used to determine the effectiveness of a method. Each group will be given 125,000 optimizer "steps" to converge on their final solution, and each solution will be evaluated against each other in each environment to determine which solution performed better than the other.

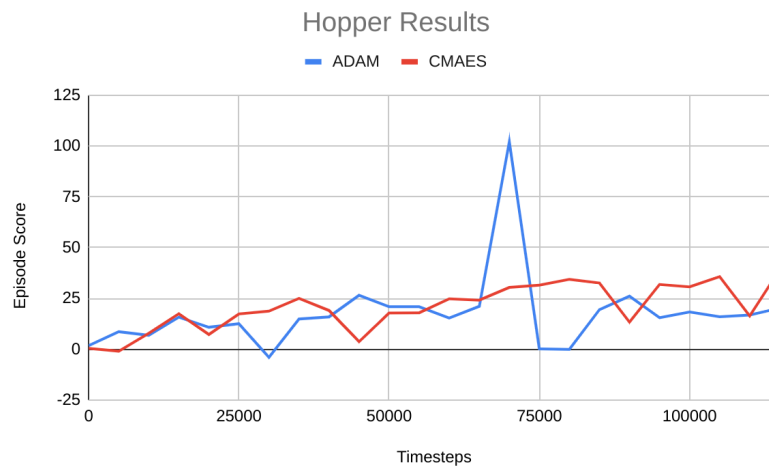
We will run 10 trials of each environment with both the covariance matrix adaptation optimizer and the Adam optimizer. From the 10 trials, we will select the trial with the best performing final solution to represent the optimizer. Then we will compare the best scores of covariance matrix adaptation against Adam in each environment. We will also be looking at the number of steps each optimizer took to converge on their solution. By convergence, I mean the point at which the optimizer stops modifying the parameters in a way that significantly affects the loss. To do this we will test the solution of each optimizer every 5000 steps, and graph the scores of each optimizer over timesteps. The point where the optimizer converges is defined by where the loss appears to be flat over time. We can plot both optimizers on 1 graph to evaluate both optimizers convergence speed and determine which optimizer converged on their solution faster.

Results

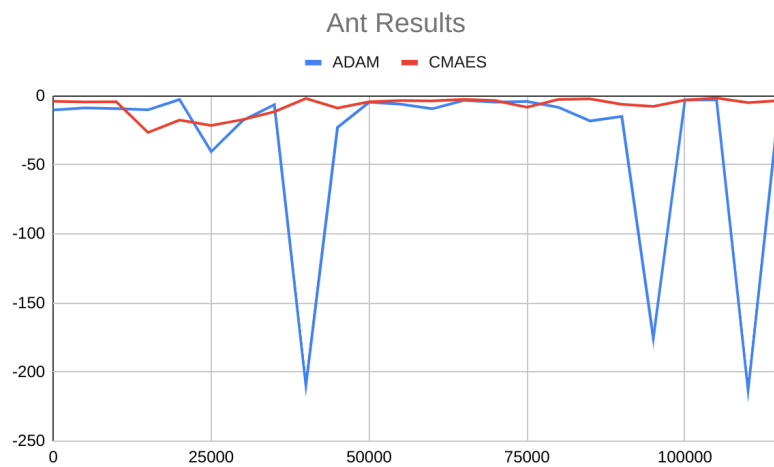
The following graphs show the results of our experiment. Each graph represents the best-performing trial of 10 trials.



Performance of ADAM and CMAES on the Half Cheetah Environment



Performance of ADAM and CMAES on the Hopper Environment



Performance of ADAM and CMAES on the Ant Environment

Analysis

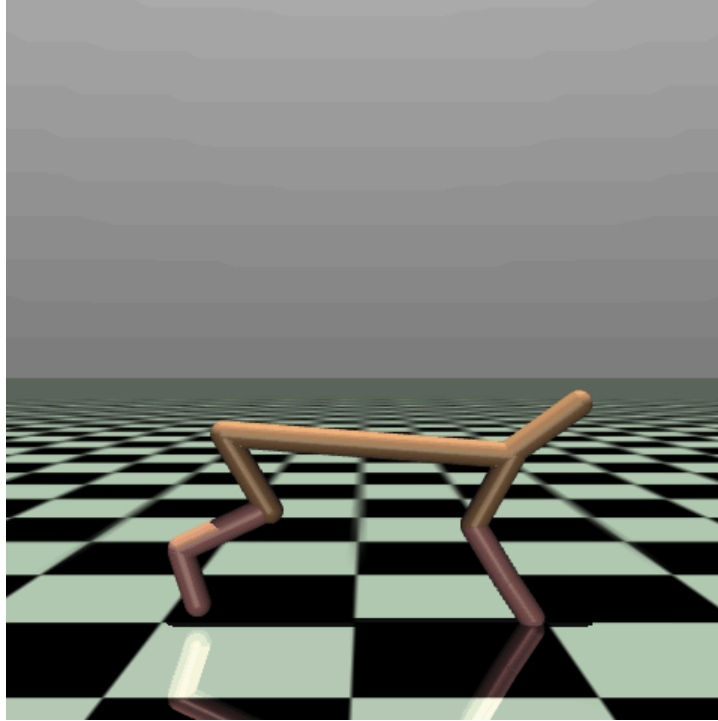
Half Cheetah (Wawrzyński, 2009)

The half cheetah problem is defined as a problem where the goal is to make the half cheetah move in the positive x direction as fast as possible. The half cheetah appears as a cat-like robot with 9 links and 8 joints. The agent can apply 2-dimensional torques to 6 of these joints, being

- the back thigh joint
- the back shin joint
- the back foot joint
- the front thigh joint
- the front shin joint
- the front foot joint

The agent observes 16 values from the half cheetah, including,

- the position of the head (3 values)
- the angle of the head (1 value)
- the angle and angular velocity of each thigh joint (4 values)
- the angle and angular velocity of each shin joint (4 values)
- the angle and angular velocity of each foot joint (4 values)



Rendering of a half cheetah

The results of the half cheetah show that for the most part, both algorithms converge on a solution at about 45,000 to 50,000 timesteps. It does appear that Adam does converge slightly faster, however, this difference by itself isn't large enough to be considered statistically significant. It also appears that covariance matrix adaptation's solution slightly outperforms Adam.

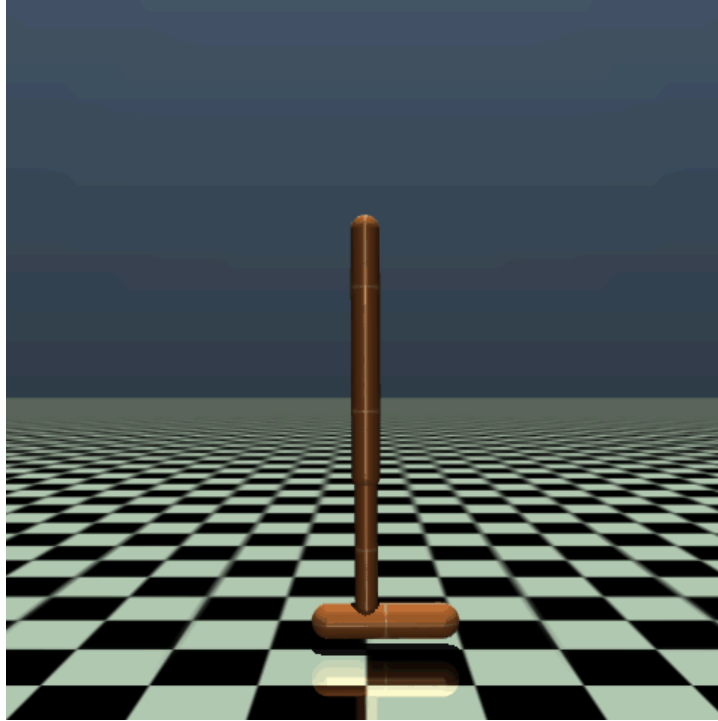
Hopper (Erez et al., 2011)

The hopper problem is defined as a problem where the goal is to make the hopper move in the positive x direction as far as possible without falling over. The hopper is a 1 legged creature with 4 distinct body parts, the torso, thigh, leg, and foot. The hopper is a unique environment because each training episode is "terminated early" when the hopper falls over, while as with regular environments, training episodes are a fixed length. The agent can apply torques to 3 joints,

- the thigh joint
- the leg joint
- the foot joint

The agent observes 11 values from the hopper, including,

- the position of the current height of the hopper (1 value)
- the angle and angular velocity of the torso (2 value)
- the angle and angular velocity of the thigh joint (2 value)
- the angle and angular velocity of the leg joint (2 value)
- the angle and angular velocity of the foot joint (2 value)
- the upwards and forward linear velocity of the torso (2 values)



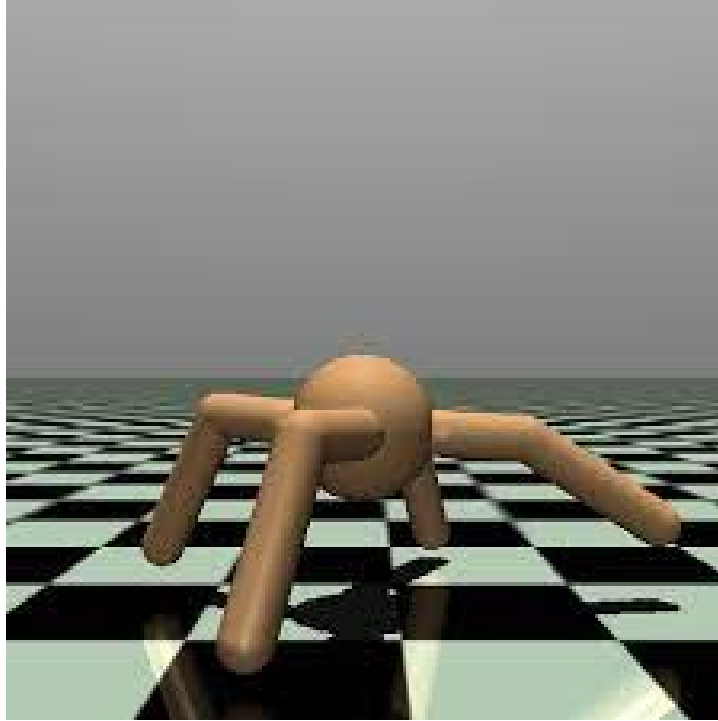
Rendering of a hopper

The results of the hopper show that both optimizers are severely underperforming in this problem. I speculate that this might be related to my specific implementation of the problem, however I cannot be certain. From this data, we can see that covariance matrix adaptation was able to slightly outperform Adam with the exception of one random data point that I believe occurred by random chance due to the noise generated by the environment. In this environment I believe these results by themselves aren't statistically significant enough to show there is a difference between the performance of covariance matrix adaptation and Adam.

Ant (Schulman et al., 2015)

The ant problem is the only 3-dimensional problem we will test, where the goal is for the ant to walk in the positive x direction as far as possible without falling over or flipping over. The ant has one torso with 4 legs attached to it. The ant has the same unique early termination condition as the hopper, where if the ant falls over or flips over the training episode is terminated early. The agent can apply 8, 3 dimensional torques, one to each joint between each hip and thigh, and one to each joint between each thigh and the leg. The agent makes very large observations consisting of 27 values, including,

- The orientation of the torso (5 values)
- The angles of the hip-thigh joints (4 values)
- The angles of the thigh-leg joints (4 values)
- The velocity of the torso (3 values)
- The angular velocity of the torso (3 values)
- The angular velocity of the torso-thigh joints (4 values)
- The angular velocity of the thigh-leg joints (4 values)



Rendering of an ant

The results of the ant show that both optimizers are severely underperforming in this problem. I believe that this is due to the limited network topology I used (more info in **Limitations**). From this data, we can see that both optimizers struggled to achieve a positive score, and I wouldn't classify either optimizer as better than the other from this data. We can see the same performance spikes in Adam that we saw in the hopper environment, and we will also attribute this to random chance but proceed with caution.

Conclusion

Limitations

Before I conclude the research paper with my findings, I want to address the limitations of my research. The original research experiments that I had cited had all been run on massive supercomputers with large network topologies for their actor and critic models, and much more than 125,000 steps. Unfortunately, I do not have access to those supercomputers, so I had to greatly scale down my research. This greatly affects the final performance of the reinforcement learning agents, and is why I believe that both optimizers performed so poorly in the ant environment. Interpret all results and conclusions from this research with this in mind, and consider how my results may change if you decide to scale up my research.

Conclusion

I wanted to evaluate the effectiveness of covariance matrix adaptation in reinforcement learning problems and compare how they perform against traditional gradient-based optimizers like Adam. From 2 of our experiments, we can see that covariance matrix adaptation slightly outperforms Adam, and from one experiment we see that covariance matrix adaptation can match Adam's level of performance. From this data, I conclude that covariance matrix adaptation is competitive with Adam in actor-critic reinforcement learning.

Future Research Direction

We can now see that covariance matrix adaptation has performance that is competitive to the performance of Adam in reinforcement learning. This opens up many new future research possibilities that were not possible before in the usage of evolutionary strategies in reinforcement learning. A few are listed below.

Quality Diversity. Quality diversity problems are a subset of reinforcement learning problems where the goal is to find multiple unique solutions to a problem rather than only one. One existing solution to quality diversity problems is MAP-Elites (Mouret & Clune, 2015), in which a brute force algorithm is used to test all possible solutions across the problem, and then store each solution in an archive. Covariance matrix adaptation, unlike Adam, searches a large area for solutions. I believe that we can combine covariance matrix adaptation with Adam to search the set of all solutions more efficiently than MAP-Elites, as we are not brute forcing all solutions but brute forcing a small search area defined by CMAES.

Vanilla Policy Gradient. As we know, Adam performs much better in actor-critic based methods than in vanilla policy gradient. This is because, in actor-critic methods, we train the actor off of the loss that was estimated by the critic. The critic's loss estimation acts as a natural smoother to the loss gradient and helps Adam traverse it more easily. However, in vanilla policy gradient, the gradients are not smooth, which results in Adam's poor performance. Since covariance matrix adaptation doesn't rely on gradients though, I believe that it could perform better than Adam in vanilla policy gradient. This research would have massive implications, as vanilla policy gradient is much cheaper computationally than actor-critic methods since vanilla policy gradient only trains one network while actor-critic methods train two.

Cluster Detection. Currently, an issue that is observable in Gaussian evolution strategies and covariance matrix adaptation is that both struggle to converge on one solution. This is because sometimes elites can be grouped in clusters, and the mean doesn't converge on either cluster but instead hovers on a sub-optimal solution between the 2 clusters. I believe this can be fixed by using cluster detection to split clusters of elites into multiple distributions controlled by multiple instances of CMAES. However, one issue is that tracking many clusters can be

expensive, so we will use the critic model to estimate the loss of the mean of each cluster, and we will stop tracking clusters with high losses. This is intended to cull out low-performing clusters of elites while helping the high-performing clusters of elites converge on a solution faster.

References

- Agarap, A. F. (2018). Deep Learning using Rectified Linear Units (ReLU) (Version 2). arXiv. <https://doi.org/10.48550/ARXIV.1803.08375>
- Erez, T., Tassa, Y., & Todorov, E. (2011). Infinite-Horizon Model Predictive Control for Periodic Tasks with Contacts. *Robotics: Science and Systems VII*. <https://doi.org/10.15607/rss.2011.vii.010>
- Fujimoto, S., van Hoof, H., & Meger, D. (2018). Addressing Function Approximation Error in Actor-Critic Methods (Version 3). arXiv. <https://doi.org/10.48550/ARXIV.1802.09477>
- Hansen, N., & Ostermeier, A. (2001). Adapting arbitrary normal mutation distributions in evolution strategies: the covariance matrix adaptation. In *Proceedings of IEEE International Conference on Evolutionary Computation*. IEEE International Conference on Evolutionary Computation. IEEE. <https://doi.org/10.1109/icec.1996.542381>
- Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization (Version 9). arXiv. <https://doi.org/10.48550/ARXIV.1412.6980>
- Mouret, J., & Clune, J. (2015). Illuminating search spaces by mapping elites. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1504.04909>
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324. <https://doi.org/10.1109/5.726791>
- Pugh, J. K., Soros, L. B., & Stanley, K. O. (2016). Quality Diversity: A New Frontier for Evolutionary Computation. In *Frontiers in Robotics and AI (Vol. 3)*. Frontiers Media SA. <https://doi.org/10.3389/frobt.2016.00040>

Ramachandran, P., Zoph, B., & Le, Q. V. (2017). Searching for Activation Functions (Version 2). arXiv. <https://doi.org/10.48550/ARXIV.1710.05941>

Robbins, H.E. (1951). A Stochastic Approximation Method. *Annals of Mathematical Statistics*, 22, 400-407.

Salimans, T., Ho, J., Chen, X., Sidor, S., & Sutskever, I. (2017). Evolution Strategies as a Scalable Alternative to Reinforcement Learning. arXiv [Stat.ML]. Retrieved from <http://arxiv.org/abs/1703.03864>

Schulman, J., Moritz, P., Levine, S., Jordan, M. I., & Abbeel, P. (2015). High-Dimensional continuous control using generalized advantage estimation. arXiv (Cornell University). <http://export.arxiv.org/pdf/1506.02438>

Statista Market Insights. (2023). Machine Learning - Worldwide | Statista Market forecast. <https://www.statista.com/outlook/tmo/artificial-intelligence/machine-learning/worldwide#market-size>

Wawrzyński, P. (2009). A Cat-Like robot Real-Time learning to run. In *Lecture notes in computer science* (pp. 380–390). https://doi.org/10.1007/978-3-642-04921-7_39

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. In *Machine Learning* (Vol. 8, Issues 3–4, pp. 229–256). Springer Science and Business Media LLC. <https://doi.org/10.1007/bf00992696>